

SASS

#sass

Preprocesadores

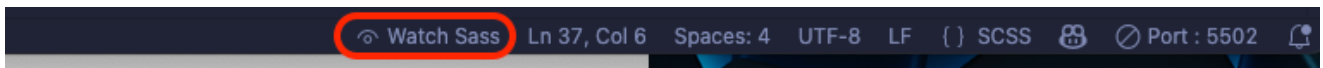
Son herramientas que nos permiten escribir código con funcionalidades extra que CSS no nos aporta, por ejemplo: *variables*, *funciones*, *operadores*, *anidación*...

Por lo que nos permite usar todas estas funcionalidades extra, para luego convertirlo en CSS estándar → de tal manera que pueda ser interpretado por el navegador

Los preprocesadores más usados, son: **Sass** y **Less**

¿cómo aplicar un preprocesador, cómo definirlo?

No defines un preprocesador dentro de una clase como tal, sino que usas su sintaxis para escribir las clases CSS de forma más estructurada, cómoda y más inteligente → esto implica que el uso de preprocesador, para esa conversión, es necesario un compilador *Live Sass Compiler* ⇒ y posteriormente tendremos que darle al botón (si usamos Visual Studio) **watch sass**



variables

una única definición del valor que podremos usar en muchos sitios

```
$color-primario: #ff5733;
$redondeado: 8px;

.mi-tarjeta {
  background-color: $color-primario;
  border-radius: $redondeado;
}
```

anidación - nesting

En lugar de repetir el nombre de la clase padre, metes los hijos dentro

```

.contenedor-padre {
  padding: 20px;

  .titulo-clase {      // Esto equivale a .contenedor-padre .titulo-clase
    font-size: 2rem;
    color: blue;
  }

  &:hover {            // El '&' referencia al padre (.contenedor-
    padre:hover)
    background: grey;
  }
}

```

```

//-----MENU-ANIDACION / NESTING-----
ul{
  list-style: none;
  padding: 0;
  margin: 60px 0;

  li {
    display: inline-block;

    a {
      text-decoration: none;
      color: white;
      padding: 20px;
      background-color: darkblue;
    }
  }
}

```

mixing -- funciones --

```

@mixin flex-centro {
  display: flex;
  justify-content: center;
  align-items: center;
}

.boton-login {
  @include flex-centro; // "Importa" los estilos del mixin aquí
  width: 100px;
}

```

MIXING CON PARAMETROS

```

@mixin boton($color) {
  background-color: $color;
  padding: 10px;
  color: white;
}

.btn-rojo {
  @include boton(red);
}

.btn-verde {
  @include boton(green);
}

```

parciales

estrategia de organizacion de archivos - y evitar que compile todos los archivos de scss que creemos

Lo definiremos como `_[nombre archivo].scss` → el `_` es necesario para que ignore el archivo y no lo compile hasta que tu se lo indiques

- nunca podra haber un archivo parcial, sin haber una hoja de estilos principal, main

archivo parcial

```

// _boton.scss
.btn-primario {
  background: blue;
  padding: 10px;
}

```

archivo principal - este es el que se va a compilar

```

// estilos-main.scss
// Importamos los parciales
@use 'variables';
@use 'botones';

body {
  font-family: Arial;
}

```